What Can Be Enforced? Limits of Runtime Guardrails for Tool-Using LLM Agents

Shawn Ray

Abstract—LLM agents rely on runtime guardrails, judge-driven gates before irreversible tool calls, against prompt injection, yet evaluations report violation rates, not what gates enforce or certify. We delimit three boundaries. Expressiveness: relative to oracle predicates, a non-retracting gate enforces exactly the nonempty safety policies whose good prefixes its register automaton recognizes, below edit automata. Statistics: Neyman–Pearson gives the exact miss/false-block frontier of the gate's information and conformal risk control a finite-sample certificate; judge-input rewrites add a robustness margin, and choosing the violation voids both; once blocks change later proposals, only exploratory logs identify the closed-loop frontier. Decidability: whether a policy can fire is undecidable with two zero-testable counters, PSPACE-complete for a keyed-counter fragment, and NP-complete for caps, quotas, and revocation. On AgentDojo, deterministic gates calibrated on 17–18 instruction-string violations to a 10% miss target block at least 64% of compliant instructions; given the user request and tool outputs of replayed traces, hosted judges separate injected from compliant calls in the same context (AUC .87–1.00) and block at most 16% of the latter, yet rewriting injected text evades them on up to 12% of flagged calls. Certificates must state the oracle interface, operating point, calibration population, adversarial margin, and feedback model.

Index Terms—AI agent security, runtime enforcement, guardrails, prompt injection, security automata, conformal risk control, large language models.

1 Introduction

Language-model agents act through tools with irreversible effects (email, payments, file edits) while reading untrusted content, so an attacker who controls that content can steer the next tool call [1]. A common defense is a runtime guardrail: a judge, often itself a language model, scores each proposed call, and a gate blocks, rewrites, or escalates it before it executes [2–8]. This gate is the security boundary between an injectable agent and irreversible effects (Figure 1). In the dependability taxonomy of Avižienis et al. [9], the gate is an error-detection and containment mechanism in front of irreversible service: each miss lets a security or safety failure through, and each false block is an omission failure that costs availability. Whether the gate can justifiably be trusted therefore depends both on its error rates and on the fault assumptions under which they are claimed.

Benchmark violation and false-positive rates do not say which policies a gate can enforce at all, what a calibrated "certificate" guarantees, or when that guarantee stops holding. For programs, the first question has a classical answer: execution monitors enforce only safety properties [10], and edit automata enforce more by suppressing and reinserting actions [11]. Agent gates depart from that setting in four ways: a fallible judge decides the policy predicate ("does this body contain PII?"); effects are irreversible, so the gate cannot buffer and retract as an edit automaton does, which returns it to the execution-monitor (safety) regime of Schneider [10]; actions carry unbounded data, which security automata already admit over countable alphabets but which a deployed gate must represent finitely; and the attacker controls part of what the judge sees. The safety half of our first result is therefore classical; its contribution is the effect-trace, oracle-relative, register-automaton form: characterizing the class a monitor with restricted state enforces is classical over finite alphabets [12, 13], and we carry it to keyed-counter register automata over infinite data alphabets, relative to oracle bits. Each result states where it applies. "Certified" means streamwise enforcement relative to fixed oracle predicates, or a marginal risk bound under explicit stochastic assumptions, never universal semantic safety.

Example 1 (Running example: an email/payments assistant)**.** An assistant acts for a user through tools such as send_email(*to*, *body*) and transfer(*to*, *amount*), under three policies: (P1) never email a body containing PII to a recipient not on the approved list; (P2) never let cumulative transfer to any recipient exceed cap c in a day bucket set by a trusted clock; (P3) after the user revokes access, send nothing further. P2 and P3 are oracle-checkable data-carrying properties; P1 rests on the semantic predicate "contains PII," which only a fallible judge evaluates.

Three boundaries: *T1, expressiveness (§4).* Relative to fixed oracle predicates, a deterministic gate enforces exactly the nonempty safety policies whose good prefixes its register automaton recognizes (Theorem 1), strictly fewer than edit automata (Proposition 1): a gate that cannot retract must refute violations on finite prefixes. P2 and P3 are inside this class; P1 needs a judge. *T2, statistics, adversaries, and feedback (§5).* With a judge, Neyman–Pearson gives the exact miss/false-block frontier of the gate's information (Proposition 2) and conformal risk control a finite-sample certificate (Theorem 3) that may be block-all; rewrites of the judge's input within budget ρ add a margin (Theorem 4); once blocks change later proposals, static scores and ungated logs cannot identify the closed-loop frontier (Proposition 3), whereas exploratory blocking logs can, at a variance that may grow exponentially in the horizon (Proposition 4). *T3, decidability (§6).* Whether a rule can ever fire is undecidable with two zero-testable counters (Proposition 6), PSPACE-complete for a keyed-counter fragment with data registers, and NP-complete for caps, quotas, and

• *S. Ray is with Carnegie Mellon University, Pittsburgh, PA 15213, USA. E-mail: shawnray@cmu.edu*
An earlier version of this work is available as arXiv:2607.22868.

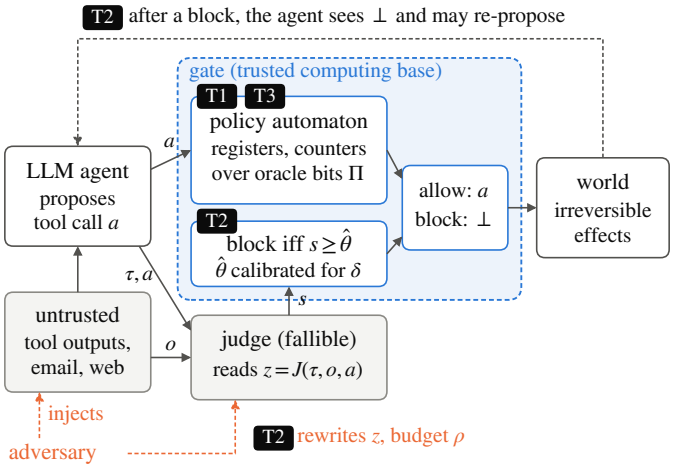


Fig. 1. A guarded tool-using agent. The agent, partly steered by untrusted input, proposes a data-carrying tool call a. The gate (the trusted computing base) combines a policy automaton over oracle bits Π with a threshold test on a fallible judge's score s of the judge-visible input z, and either commits a irreversibly or blocks it by committing the no-op $\perp$. T1: what the automaton can enforce. T2: what a judge-based test can certify (frontier, calibration of $\hat{\theta}$, budget-ρ rewrites of z, feedback). T3: whether a rule can ever fire, decided before deployment.

revocation without them, so one SMT query lints them (Theorem 5). Table 1 gives each result's regime and what is imported versus new.

Empirical findings: On AgentDojo [16] (§7), with 33 judge runs (32 models):

- *Instruction strings make gates expensive.* After familywise correction only Claude Opus 5.5 (AUC .929, refusals counted as blocks; not if scored .5) beats Gemma-2-9B (AUC .813) in all three runs. At miss target $\delta = .10$, with 17–18 calibration violations per fold, even the best deterministic split-conformal gate blocks 64% of compliant instructions (95% CI 20–83%; 11% without the 7 goals whose text is benign), because $\hat{\theta}$ is then the single lowest violating calibration score, and every $\delta = .05$ gate is block-all; exact-level randomized calibration halves the cost (33%).
- *What the gate sees decides the frontier.* Given the user request and the tool outputs, the seven hosted judges separate injected from compliant calls in the same context with AUC .872–.999 (.621–.835 for the same calls serialized alone), and gates cross-fitted over goals miss .038–.098 while blocking at most 5.3% of clean compliant calls (43.7–92.6% without context).
- *Context buys discrimination, not verified robustness.* In context, best-of-20 rewrites of the injection stably evade the $\delta = .10$ gate on 12 of 101 flagged calls for Claude Haiku 5.5 net of a clean-prompt null control, and on at most 6 for each other judge. On instruction strings, one predeclared 16-word suffix raises the miss rate at $\theta = .5$ by .200–.657 for 10 of 21 judges (no Claude or GPT judge), and an optimizing attacker evades even the gates that block 32–87% of compliant instructions on 27–35% of flagged violations. Attacks refute margins; they never certify one.
- *Closed loop and cost.* A 3B-judge gate that drops 78% of a local 7B agent's calls cuts attack success from .047 to .004 (descriptive; no A/A or random-drop control), and a compiled P2+P3 gate decides in under a microsecond, about 10^6 times faster than a hosted judge call (§7.6).

A guardrail certificate therefore means little without its oracle interface, operating point, calibration population, adversarial margin, and feedback model (§9).

Contributions: Beyond the imported ingredients named in Table 1:

- *An exact enforceability boundary* (§3–§4): the oracle/judge split, the class a non-retracting register gate enforces over data-carrying effects, and a separation showing why the oracle interface must be action-local.
- *An attack and feedback accounting for calibrated gates* (§5): benign calibration degrades to $\delta + H\bar{\eta}(\rho)$, margin-aware recalibration restores δ only for a verified margin, and closed-loop frontiers are identified by exploratory blocking but not by static scores or ungated logs.
- *A decidable policy fragment* (§6): nontriviality is PSPACE-complete for the fragment (hard via registers alone) and NP-complete for caps, quotas, and revocation without registers.
- *An evaluation* (§7) that separates calibration cost (33 judge runs), what the gate sees (tool calls with and without context), representation attacks, and closed-loop effects.
- *A guardrail certificate card* (§9, Table 2): five fields every evaluation must report, each required by one of our results, with a template in the artifact.

Code, the curated probe, and cached judge scores are public[1] (except the source-role and closed-loop-rerun caches; Supplement C); complete proofs and extended experiments are in the supplementary material (Supplements A–E).

2 System and Threat Model

System and trusted computing base: An agent acts for a user by proposing tool calls one at a time. A pre-execution gate sees the committed effect history, the proposed action a, and the scores of m judges, and returns allow, block, or sub(a') before the action commits (Def. 4). Commits are irreversible; blocking commits the no-op $\perp$, which has no external effect. The agent may observe a block and change its later proposals; T1 abstracts this feedback away and T2 models it as a decision-dependent kernel (Def. 5). The TCB consists of the gate code, its policy automaton, the oracle predicate bits Π when available, and trusted metadata (in the running example, a trusted clock adapter for P2's day bucket and provenance labels that grant authority only to user input). The agent is outside the TCB, so T1 quantifies over every proposal stream; the judge is a black-box test with false-alarm rate α and miss rate β (Def. 3). In the terminology of Avižienis et al. [9], injected content and judge errors are faults, a violating proposal that reaches the gate is an error, a violating committed effect is a failure, and a false block is an omission failure.

Adversary: The adversary controls untrusted content the agent reads (tool outputs, documents, email) and uses it for indirect prompt injection [1]. It has three capabilities. (i) *Injection:* it chooses the injected content, and hence which violating proposals the agent makes. (ii) *Knowledge:* it knows the policy, the scoring function, and the threshold. (iii) *Judge-input rewriting* (Def. 6): at deployment, adaptively across steps, it may replace the judge-visible representation $z = J(\tau, o, a)$, with τ the committed effect history and o the untrusted context the judge reads, by any z' with $d(z, z') \leq \rho$, e.g., by inserting at most ρ tokens into a tool-output field, leaving the action, its effect, and its label unchanged;

1. https://github.com/shawnray-research/certified-agent-guardrails/tree/tdsc

TABLE 1
The three boundaries at a glance. Each result holds only in its stated regime; the evidence column points to the experiment that probes it.

Boundary and result	Regime	Imported ingredient → what is new	Consequence for deployments	Evidence
T1 expressiveness (Thm. 1, Prop. 1): a deterministic gate enforces exactly the nonempty safety policies with $\mathrm{Good}(P) \in \mathsf{RA}[\mathcal{T}; \Pi]$, strictly fewer than edit automata	fixed oracle bits Π; irreversible commits; every proposal stream	security, truncation, and bounded-memory automata [10, 11, 13] → effect-trace, oracle-relative, keyed-counter register form over data-carrying actions	compile caps, quotas, and revocation to exact gates; deferred commits belong in the tool interface, not in the gate	§7.6
T2 exogenous frontier (Prop. 2, Cor. 1): the ROC of the gate's information is the exact miss/false-block frontier	one-step law independent of the gate	Neyman–Pearson, Blackwell → the object guardrail benchmarks should report, lifted to episodes	no ensemble or wrapper on the same information beats it; report an operating point, not AUC	§7.2
T2 certificate (Thm. 3): calibrated per-step miss risk δ/H bounds the unsafe-episode rate by δ	label-conditional exchangeability (Asm. 1)	conformal risk control → per-step gate calibration on violating examples	the bound is marginal and may be met only by blocking everything	§7.2
T2 adversary (Thm. 4, Prop. 5, Cor. 3): the bound degrades to $\min\{1, \delta + H\bar{\eta}(\rho)\}$	benign calibration; budget-ρ judge-input rewrites (choosing the violation voids it)	robust conformal prediction [14] → risk-level margin-aware recalibration for discrete rewrites, valid once $\bar{\eta}(\rho) < \delta/H$ is verified	benign calibration is not a security boundary; attacks can refute, never certify, a margin	§7.4
T2 feedback (Prop. 3, Cor. 2, Prop. 4, Thm. 2): ungated logs do not identify the closed-loop frontier; exploratory blocking does	blocks change later proposals	CPOMDP occupancy, off-policy overlap → non-identifiability for gated agents; occupancy program for finite models	evaluate in the loop, logging known block and allow probabilities; sample cost can grow like ε^{-H}	§7.3, §7.5
T3 decidability (Prop. 6, Thm. 5): whether a policy can ever fire is undecidable with two zero-testable counters, PSPACE-complete for $\mathcal{T}_0$, NP-complete for caps, quotas, revocation	static analysis of the policy specification	Minsky machines, register-automaton PSPACE-hardness [15] → $\mathcal{T}_0$ via exact cap saturation and a one-witness reduction; Parikh phases for cap checks	keep policy languages inside $\mathcal{T}_0$ (caps, quotas, revocation: one SMT query) to ship a complete "can this rule fire" linter	§7.6

since tool outputs are attacker-written, ρ is limited in practice only by the field length, so a margin must be stated for the budget a deployment admits. Calibration examples are benign. Theorem 4 prices only (iii): it requires that the violating proposals produced under (i) remain label-conditionally exchangeable with the calibration violations (Assumption 1), and an adversary that uses (i) to choose or shift the induced violation, e.g., toward goals whose text reads as benign (§7.2), voids that assumption and with it Theorems 3 and 4. Our experiments (§7.4) instantiate only black-box members of (iii), which lower-bound $\bar{\eta}(\rho)$: they can refute a margin but never certify one. A deployed gate should treat a refusal or any other non-answer as a block (fail-closed); §7 states how our caches score refusals. Out of scope are compromise of the gate, the oracle, or the trusted adapters; training-time poisoning of the judge or of the calibration set; side channels; and adversarially induced false blocks (a denial-of-service lever our bounds do not price).

Goals and what "certified" means: The gate is a reference monitor for tool calls [17]: we assume complete mediation (every external effect is a typed tool call through the gate, which excludes generic code-execution tools) and a tamperproof TCB, and T3 addresses the third requirement, verifiability of the policy; blocking is the fail-safe default [18]. "Certified" has exactly two senses. The first is streamwise enforcement relative to fixed oracle predicates: a gate is sound if every committed trace is safe for P and transparent if it never alters a compliant stream (Def. 4), for every proposal stream. The second is a marginal risk bound: under Assumption 1, a calibrated gate bounds the probability of an unsafe episode by δ, marginally over episodes and the calibration draw (Theorem 3); against the budget-ρ adversary the bound becomes $\min\{1, \delta + H\bar{\eta}(\rho)\}$ (Theorem 4), and margin-aware recalibration restores δ once a uniform margin $\bar{\eta}(\rho) < \delta/H$ is verified, at a cost in false blocks (Corollary 3). The availability goal mirrors soundness: transparency in the oracle regime, and a reported false-block rate at the certified miss level in the judge regime. Universal semantic safety is a non-goal.

3 The Enforcement Model

We formalize the objects of §2; Supplement A.1 collects the notation.

Definition 1 (Actions, data, traces)**.** Fix finite action types Σ with arities $\mathrm{ar} : \Sigma \to \mathbb{N}$ and a data domain $\mathcal{D}$ (possibly infinite and many-sorted, e.g., an equality sort plus an $\mathbb{N}$ amount sort; with equality, optionally a linear order). An *action* is $a = (\sigma, \vec{d})$ with $\vec{d} \in \mathcal{D}^{\mathrm{ar}(\sigma)}$. The alphabet $A = \bigcup_\sigma \{\sigma\} \times \mathcal{D}^{\mathrm{ar}(\sigma)}$ is infinite when $\mathcal{D}$ is and some arity is positive. Traces are words in A^* (finite) or A^ω (infinite); $\preceq$ is the prefix order. A distinguished *no-op* $\perp \in A$ has no external effect, and the *external-effect projection* $\mathsf{ext} : A^\infty \to (A \setminus \{\perp\})^\infty$ deletes every $\perp$. We score a committed trace c on its *effect* trace: c is *safe* for P iff (i) no finite prefix of $\mathsf{ext}(c)$ is a bad prefix of P and (ii) $\mathsf{ext}(c) \in P$ whenever $\mathsf{ext}(c)$ is infinite. For safety P (Def. 2), (ii) follows from (i) (Supplement A.1), so only bad prefixes matter, and committing $\perp$ (blocking) is a safe default that never adds one.

The no-op abstracts external effects only: T1 ignores a block's latency, retries, and memory side effects; when these change later proposals, T2 models them (Def. 5).

Definition 2 (Safety, register representation)**.** A policy is a set $P \subseteq (A \setminus \{\perp\})^\omega$ of infinite *effect* traces (for policies, A^* and A^ω below range over the effect alphabet $A \setminus \{\perp\}$). P is *safety* [19] iff every $x \notin P$ has a finite bad prefix: $\exists n \, \forall y : \; x_{\le n} y \notin P$. Let $\mathrm{Bad}(P) \subseteq A^*$ be the minimal bad prefixes and $\mathrm{Good}(P) = A^* \setminus \mathrm{Bad}(P) A^*$ the (prefix-closed) good prefixes. Our policy-automaton model is a *keyed-counter register automaton* (a custom class): it extends the register automaton of Kaminski and Francez [20] with *finite-support key maps*. Concretely it has finite control, finitely many data registers

over $\mathcal{D}$ (their number is part of the specification; distinct registers may hold equal data), finitely many finite-support key maps, each sending a key (a tuple of data values named by a designated field of the current action) to a per-key finite local state and a vector of per-key counters over $\mathbb{N}$ (unattached keys default to the initial local state and 0; a transition reads and updates only the entries of the keys the current action names), and, when declared, one fixed finite vector of global $\mathbb{N}$-counters. Numeric counters are distinct from data registers; transitions choose their successor and updates by $\mathcal{T}$-guards on the current action's data, the registers, and the counters, with absorbing bad states. Counter values are unbounded in general; the guard theory $\mathcal{T}$, including its allowed counter updates, sets expressivity (the separable, key-local, constant-guard monotone fragment $\mathcal{T}_0$ is decidable; $\mathcal{T}^*$ with two decrementable zero-test counters is undecidable; §6). At any finite time only finitely many keys are active, so the recognizer stays deterministic and finitely represented. $\mathsf{RA}[\mathcal{T}]$ is the class of finite-word languages such an automaton recognizes; P is *$\mathcal{T}$-representable safety* if it is safety and $\mathrm{Good}(P) \in \mathsf{RA}[\mathcal{T}]$. When guards may query a fixed finite vector of externally supplied predicate bits Π, we write $\mathsf{RA}[\mathcal{T};\Pi]$ and say *representable relative to* Π; this interface does not grant access to any other semantic fact.

For P2 of Example 1, a trusted clock adapter tags each transfer to recipient k of amount v with a day bucket w, written $\mathsf{transfer}(k, v, w)$; the automaton keeps one monotone counter per key (k, w) and tests $spent_{k,w} + v \le c$. The class is not tuned to the theorem: over an infinite recipient sort with equality only, registers and global counters without key maps cannot even express a per-recipient cap of one, for any action-local Π (Proposition 1(ii); cf. [20]); unrestricted counters are undecidable (§6); and monotone per-key counters match deployed caps, quotas, and rate limits. We take specification automata to be trim (every reachable non-bad configuration has some non-bad successor under an effect action, e.g., an always-allowed read), so that the language of words that avoid the bad states is exactly $\mathrm{Good}(P)$ and "block iff the next state is bad" is the T1 gate. Trimness is a hypothesis on the specification, not something the gate checks: an always-allowed action guarantees it, but for $\mathcal{T}^*$ it is undecidable in general (by a variant of the reduction behind Proposition 6).

Definition 3 (Predicates, judge, error rates)**.** Guards are boolean combinations of $\mathcal{T}$-constraints on data and semantic predicates $\pi_i : (A \setminus \{\bot\})^* \times A \to \{0,1\}$ evaluated on the committed effect history $\tau \in (A \setminus \{\bot\})^*$ (ground truth). A *judge* $\hat{\pi}_i^{\theta}(\tau, a) = \mathbf{1}[s_i(\tau, a) \ge \theta_i]$ approximates π_i via a score s_i; under a distribution $\mathbb{P}$ on (τ, a) its false-alarm and miss rates are $\alpha_i(\theta_i) = \mathbb{P}[\hat{\pi}_i = 1 \mid \pi_i = 0]$ and $\beta_i(\theta_i) = \mathbb{P}[\hat{\pi}_i = 0 \mid \pi_i = 1]$. The *oracle* case is $\hat{\pi}_i = \pi_i$ on every (τ, a) (not merely $\alpha_i = \beta_i = 0$ under one $\mathbb{P}$) for the fixed available vector $\Pi = (\pi_1, \ldots, \pi_m)$; Π is *action-local* if each $\pi_i(\tau, a)$ depends on a alone (including trusted metadata carried as data on a). The *ROC frontier* $\mathrm{roc}(\beta)$, a nonincreasing convex function of β, is the least false-alarm rate attainable at miss rate β by any randomized decision measurable with respect to s_i (traced by likelihood-ratio tests on s_i, and by thresholds when s_i is monotone in the likelihood ratio).

Formal agent policies with fallible predicates, such as a PII detector inside a Datalog policy, also appear in Palumbo et al. [21] and Solko-Breslin et al. [22]; we separate the oracle and judge regimes so that each gets an exact boundary. A history-dependent bit such as $\mathbf{1}[\tau \cdot a \in \mathrm{Good}(P)]$ would make every safety policy representable, so our instantiations use action-local Π ("this body contains PII") and carry history-dependent facts such as provenance as data on actions.

Definition 4 (Gate, soundness, transparency, utility)**.** Taking $\mathcal{D}$ countable and effectively presented and scores finitely represented (e.g., rational), so that computability is meaningful, a *gate* is a computable map $g : (A \setminus \{\bot\})^* \times A \times [0,1]^m \to \mathrm{Dec}$ (the m judge scores; the oracle case of T1 restricts the last argument to $\{0,1\}^m$ and lets g be computable relative to Π, reusing bits returned at earlier steps), $\mathrm{Dec} = \{\mathsf{allow}, \mathsf{block}, \mathsf{sub}(a')\}$, deciding before the (irreversible) action commits from the committed *effect* history (blocks leave no effect to condition on), the proposed action, and the scores. Against a proposal stream it induces a committed trace (allow keeps a; block commits $\bot$; $\mathsf{sub}(a')$ commits a'). g is *sound* if for *every* stream the committed trace is safe for P (Def. 1: no finite prefix of its effect trace is a bad prefix of P, and an infinite effect trace lies in P); *transparent* if for every stream whose *proposed* effect trace lies in P the committed effect prefix equals the proposed effect prefix *at every step* (prefixwise, so no action of a compliant trace is ever blocked or altered); *precise* if both (precise enforcement in the sense of Ligatti et al. [11]: no buffering). For horizon H and distribution $\mathbb{P}$, an episode is *unsafe* if some committed effect prefix is a bad prefix of P, and the *utility loss* is $U(g) = \frac{1}{H}\sum_{t=1}^{H} \mathbb{P}[g \text{ alters } a_t \mid \text{the proposed } a_t \text{ is compliant}]$, the per-step rate, where a_t is *compliant* if the committed effect history extended by a_t is a good prefix of P (oracle regime) or its ground-truth predicate is 0 (judge regime, Def. 3).

4 T1: The Expressiveness Boundary (Oracle Enforceability)

Call g a $(\mathcal{T};\Pi)$-*gate* if it runs a deterministic keyed-counter register automaton (Def. 2) with $\mathcal{T}$-guards and queries to the fixed predicate bits Π once, online and in commit order, over the committed *effect* history, and decides each proposal by the one transition it takes from the configuration reached, labeled allow, block, or $\mathsf{sub}(t)$ for a term t over the proposal's fields, the registers, and constants; Remark 1 treats general computable gates.

Theorem 1 (Oracle characterization; classical [11, 23], instantiated)**.** *Fix any oracle interface Π (Def. 3; with action-local Π the bits cannot replace the automaton's memory, Proposition 1(ii)) and a nonempty policy P. Then P is precisely enforceable by a deterministic $(\mathcal{T};\Pi)$-gate if and only if it is safety and $\mathrm{Good}(P) \in \mathsf{RA}[\mathcal{T};\Pi]$. Enforcement is witnessed by simulating that recognizer and refusing exactly the one-step extensions it rejects; each step evaluates one enabled transition in the given guard theory.*

Proof idea. ($\Leftarrow$) Simulate the recognizer on the committed history and block exactly the extensions it rejects. ($\Rightarrow$) If P is not safety, some violating x is a limit point of P; transparency on its compliant extensions forces the deterministic gate to allow every step of x, contradicting soundness. Simulating the gate on finite words yields a recognizer of $\mathrm{Good}(P)$ (Supplement A.2).

Running example: P3 needs one control flag set by revoke, and P2 one monotone counter per key (k, w) with guard $spent_{k,w} + v \le c$, so a gate enforces both without semantic predicates and never blocks a compliant send or transfer. P1's guard, "the body contains PII," is semantic: a gate enforces P1 only if that predicate is in Π; with a fallible judge in its place, T2 governs.

Remark 1 (General gates). An arbitrary computable gate with the same oracle interface enforces a nonempty P if and only if P is safety with good-prefix membership decidable relative to Π: the truncation-automaton characterization of Ligatti et al. [11, 23] (already over countably infinite action sets), with blocking in place of halting and with oracle-relative decidability. $(\mathcal{T}; \Pi)$-gates are its register-recognizable fragment.

Proposition 1 (A strict hierarchy)**.** *Give every enforcer the same oracle bits* Π. *(i) Gates, which enforce precisely, enforce strictly fewer policies than edit automata under effective enforcement [11] (an* ω*-policy counts as edit-enforceable when some extension of it to finite executions is, Supplement A.2); the gap is exactly buffering, which irreversibility removes. Witness: "every* pay *is eventually followed by a* confirm*" is a renewal property an edit automaton enforces by buffering* pay*, but transparency on the compliant streams* $\mathsf{pay}^k\mathsf{confirm}^\omega$ *forces a gate to allow every irreversible* pay*, so on* pay^ω *it commits a violation no later decision can retract. (ii) If* Π *is action-local and recipients range over an infinite equality-only data sort, then no* $(\mathcal{T}; \Pi)$*-gate without key maps (whatever its registers, global counters, and counter guards and updates) enforces the per-recipient cap of one, which a* $(\mathcal{T}_0; \Pi)$*-gate enforces with one key-local flag. Witness: after* $r+1$ *allowed sends to distinct recipients that avoid the gate's constants and share oracle bits, some recipient* d *is in none of the gate's* r *registers, and a fresh such recipient* e *satisfies exactly the same guards, although a second send to* d *is a violation and a send to* e *is not.*

Part (i) is the classical truncation/edit separation [11, 23], restated to locate gates, and holds for every Π; part (ii) is where the restriction on Π matters, since with the history bit $\mathbf{1}[\tau \cdot a \in \mathrm{Good}(P)]$ a register-free gate enforces the cap (§3). The witness of part (i) is a common design: a deferred-commit payment protocol needs buffering, so the deferral must live in the tool interface, not in the gate. Over finite alphabets, enforcers for reactive systems must release an output at every reaction, so not every safety property is enforceable there [24, 25]; our gate can always commit the no-op. Over data, determinism is a genuine restriction [26], and relational policies are hyperproperties outside this class [27, 28].

Substitution ($\mathsf{sub}(a')$) leaves the class unchanged, and so does escalation if the escalation oracle is part of Π: the limit-point argument and the never-alter recognizer apply verbatim. With an oracle, a transparent gate never alters a compliant proposal, so substitution cannot improve U; with a judge, committing a *prefix-safe* rewrite (ua' good whenever u is) instead of blocking lowers the graded false-block loss to at most $\ell_{\mathrm{sub}}\alpha(\theta)$ at unchanged miss, where $\ell_{\mathrm{sub}} \in [0, 1]$ is a rewrite's loss relative to a block's unit loss. Escalation fits only as an oracle that returns prefix-safe actions (Supplement A.2).

5 T2: The Statistical and Security Boundaries (Fallible Judge)

T2 asks what a gate can guarantee when semantic predicates are evaluated by a fallible judge (Def. 3). For the exogenous binary decision the ROC is the exact object (§5.1); in the closed loop, where blocking changes what the agent proposes next, it is not (§5.2). We then turn the exogenous frontier into a finite-sample certificate (§5.3) and bound what survives an adversary who controls the judge's view of an action (§5.4). Full proofs are in Supplement A.3.

Assumption 1 (Benign calibration regime)**.** One binary predicate labels a proposal violating iff its commit leaves Good(P); the gate is block-only; and violating calibration examples are label-conditionally exchangeable with violating deployment proposals. The product-episode specialization additionally assumes i.i.d. labels and conditionally i.i.d. information across steps, and a stationary rule with fresh per-step randomization.

Judged predicates enter the gate only as stateless conjuncts of its block rule; judged bits that update automaton state are out of scope.

5.1 The Exogenous Frontier

Fix a single pre-commit decision and let I be the information the gate sees before it decides (e.g., one or several judge scores). The gate cannot influence the law of I, so choosing a block rule is a two-class testing problem.

Proposition 2 (Exogenous information frontier; Neyman–Pearson and Blackwell, restated)**.** *Fix a one-step law for the violation label and any pre-commit information* I *whose law does not depend on the gate. Write* $\mathrm{roc}_I(\beta)$ *for the minimum false-block probability among all randomized* I*-measurable rules with violation-miss probability at most* β*. Then the exact safety–utility frontier is*

$$B_I^\star(\beta) = \mathrm{roc}_I(\beta),$$

where $B_I^\star(\beta)$ *is the minimum utility loss* U *(Def. 4) of a block-only gate that decides from* I*, possibly with external randomization, at miss at most* β*, attained by a randomized likelihood-ratio test (by a threshold on a score* s*, randomized on ties, when that ratio is a nondecreasing function of* s*). No rule, ensemble, or wrapper using the* same I *lies below this frontier. If* I' *refines* I *(so* I *is a garbling of* I'*), then* $\mathrm{roc}_{I'}(\beta) \le \mathrm{roc}_I(\beta)$ *pointwise. The two frontiers coincide at every* β *whenever the likelihood ratio of* I' *is almost surely a function of* I *(then* I *is sufficient for the label), so conditional dependence among judges alone does not guarantee a gain.*

The proof is the Neyman–Pearson lemma [29, 30] with Blackwell's garbling order [31] (Supplement A.3): a guardrail can improve on a judge only by changing the information it decides on.

Corollary 1 (Continuation-neutral episode lift)**.** *Under the product specialization of Assumption 1, restrict the gate to applying one stationary* I*-measurable test independently at each step. If every step is violating,* $R_H(\beta) = 1 - (1-\beta)^H$*; if labels are i.i.d.* Bernoulli(q)*,* $R_H(\beta) = 1 - (1 - q\beta)^H$*. Within this product class,*

$$B_{I,H}^\star(\delta) = \mathrm{roc}_I\left(R_H^{-1}(\delta)\right),$$

where $B_{I,H}^\star$ *is the minimum per-step false-block probability and* $R_H^{-1}(\delta) = \max\{\beta \in [0,1] : R_H(\beta) \le \delta\}$ *(so allow-all once* $\delta \ge R_H(1)$*). This lift does* not *cover history-dependent rules (which can do strictly better even without feedback), shared episode-level randomization, or feedback from decisions into future proposals; open-loop time-varying rules cannot beat it (Supplement A.3).*

5.2 Feedback: The Closed-Loop Frontier

Corollary 1 assumes that a block changes neither what the agent proposes next nor how later proposals are labeled and scored. A tool-using agent sees a block and may respond (e.g., by retrying), so we model the gate as the controller of a finite partially observed process.

Definition 5 (Controlled episode frontier)**.** For a finite horizon and finite abstraction, let X_t contain the environment, agent memory, policy state, and an absorbing unsafe bit. From its perfect-recall history h_t (including proposal and score), the gate chooses $D_t \in \{\mathsf{allow}, \mathsf{block}\}$; the kernel for X_{t+1} and the next observation (appended to h_t) may depend on D_t, and depends on the past only through (X_t, D_t). For bounded additive cost c_t and first-entry unsafe cost r_t, both functions of (X_t, D_t, X_{t+1}) (the current proposal is part of X_t), define $C(\psi) = \mathbb{E}_\psi \sum_t c_t$, $R(\psi) = \mathbb{E}_\psi \sum_t r_t$, and $F_H(\delta) = \inf_{\psi: R(\psi) \le \delta} C(\psi)$ over randomized perfect-recall policies.

Here the gate observes its full history h_t, including blocked proposals and past scores, which extends the observation model of Definition 4; T1 is unaffected because its arguments use only block-free streams.

Theorem 2 (Endogenous finite-model frontier; CPOMDP occupancy, instantiated)**.** *For a specified finite controlled model in Definition 5, every history-dependent policy has an equivalent belief-Markov policy. Reachable-belief occupancy measures form a finite polytope, and*

$$F_H(\delta) = \min_{\zeta \in \mathcal{M}} \sum_{t,b,d} \zeta_t(b,d) c_t(b,d) \quad \text{s.t.} \quad \sum_{t,b,d} \zeta_t(b,d) r_t(b,d) \le \delta,$$

where $b_t = \mathbb{P}(X_t \mid h_t)$, $c_t(b,d) = \mathbb{E}[c_t \mid b_t = b, D_t = d]$, $r_t(b,d)$ likewise, and $\mathcal{M}$ is the set of nonnegative occupancy measures over reachable beliefs that satisfy the flow constraints of Supplement A.3. Thus the achievable (R, C) set is compact and convex and F_H is nonincreasing, convex, and piecewise linear where finite. Strong duality reduces each supporting point to a belief-state Bellman recursion V_t^λ with stage cost $c_t + \lambda r_t$ (Supplement A.3), and, whenever the risk constraint is feasible, an optimum is implemented by sampling once per episode from at most two deterministic belief-Markov policies [32].

Proof idea. Finitely many beliefs are reachable and the belief is a sufficient controlled state; occupancy measures form a polytope on which R and C are linear [33], and LP duality yields the recursion (Supplement A.3).

The optimal rule blocks only when the immediate gain from blocking covers the future cost the block causes (e.g., a retry that must itself be blocked), so the one-step ROC governs it only where that continuation gap vanishes, as at $t = H$ (Supplement A.3).

Proposition 3 (ROC does not identify a feedback frontier)**.** *Two $H = 2$ systems can have identical judge ROC and identical ungated trajectory law but different $F_H(0)$ (minimum intervention cost at zero episode-unsafe rate): after a first-step block, system A stays compliant whereas system B proposes a fresh violation with positive probability. Hence closed-loop performance requires the controlled response to intervention, not the ungated ROC alone.*

Construction. Take a constant judge score and a first proposal that violates with probability $q \in (0,1)$. After an allow, both systems propose a compliant action, so ungated laws and ROCs coincide; after a block, A proposes a compliant action whereas B proposes one that violates with probability $p \in (0,1)$. Zero risk forces blocking, so $F_A(0) = 1 - q < (1-q) + (1-p) = F_B(0)$; the gap persists for every judge whose score laws are not mutually singular (Supplement A.3).

Corollary 2 (Identification impossibility)**.** *Over any model class containing the two systems in Proposition 3, no estimator whose input is limited to ungated trajectories and static judge scores can uniformly identify $F_H(0)$, hence cannot identify the full controlled frontier.*

The estimator's input law is the same in A and B, but the targets differ: this is the overlap (positivity) failure of off-policy evaluation [34–36], since ungated logs never exercise a block, and reruns of one fixed non-exploring gate identify only its own outcomes. Exploration restores identification, at a price that grows with the horizon.

Proposition 4 (Exploratory blocking restores identification)**.** *In the model of Definition 5, let a logging gate choose D_t from h_t with known probabilities $\mu(d \mid h_t) \ge \varepsilon > 0$ for both decisions at every reachable history, and let the logs record (h_t, D_t, c_t, r_t) at every step (for c_t, this includes the ground-truth label of a blocked proposal). Then for every randomized perfect-recall gate ψ, with $W_t = \prod_{s \le t} \psi(D_s \mid h_s)/\mu(D_s \mid h_s)$,*

$$R(\psi) = \mathbb{E}_\mu\Big[\sum_t W_t r_t\Big], \qquad C(\psi) = \mathbb{E}_\mu\Big[\sum_t W_t c_t\Big],$$

so the achievable (R, C) set and $F_H(\delta)$ are identified from the logging law. Since $W_t \le \varepsilon^{-t}$, the importance-weighted estimator is unbiased but its variance can grow exponentially in H.

Proof idea. The trajectory law factorizes into the gate's decision probabilities and gate-independent conditional laws, so per-decision importance sampling [36] applies wherever μ is positive (Supplement A.3). Ungated logs have $\mu(\mathsf{block} \mid h) = 0$, exactly the failure in Corollary 2: a gate's frontier is identified from logs once the logging gate sometimes blocks and sometimes allows at every history the evaluated gates can reach; exploratory allows of flagged proposals must therefore run in a sandbox or shadow deployment, never against real irreversible effects.

5.3 A Distribution-Free Certificate

The frontiers above are population objects; a deployment needs a guarantee for the threshold it selects from finite data.

Theorem 3 (Distribution-free safety certificate; conformal risk control, instantiated)**.** *Under Assumption 1, calibrating the threshold by conformal risk control, on violating calibration examples, to hold the per-step conditional miss risk $\mathbb{P}[\mathsf{allow} \mid \textit{violation}]$ at δ/H yields $\mathbb{P}[\textit{unsafe episode}] \le \delta$, marginally over deployment episodes and the calibration draw, finite-sample and distribution-free under per-step exchangeability. The theorem certifies risk, not proximity to the population ROC frontier; finite calibration can select a conservative or block-all threshold.*

Proof idea. Conformal risk control [37] bounds each step's conditional miss by δ/H, and a union bound over the H steps gives δ (Supplement A.3); a guarantee conditional on one calibration set needs more violations [38].

Remark 2 (On the assumptions). Exchangeability is label-conditional and per deployment proposal, not a within-episode independence claim. Assumption 1 is an open-loop condition: once decisions change later proposals, their labels, or the judge's inputs, their law depends on $\hat{\theta}$, so it must be checked for the gated loop itself (feedback is absent at $H = 1$ or for an exogenous proposal stream with history-free, action-local scoring); our closed-loop study (§7.5) does not claim it.

The factor H comes only from the union bound: under a stronger, open-loop episode exchangeability, calibrating on the episode loss removes it, and the gate that blocks when any of m judges flags has sharp Fréchet–Hoeffding error bounds (Supplement A.3).

5.4 Adversarial Deployment

An attacker who can write into the judge's input breaks the certificate's exchangeability without changing what the action does. We bound the damage by a representation margin.

Definition 6 (Budget-ρ adversary; robustness margin)**.** Let the judge score its input $z = J(\tau, o, a)$ (not just (τ, a)), where o is untrusted context the judge reads (e.g., tool outputs) that is not part of the committed effect history τ. A *budget-ρ adversary* knows the scoring function and threshold and may replace z by z' with $d(z, z') \le \rho$ (for example, inserting at most ρ tokens into a tool-output field), adaptively across steps. The executable proposal a, its effect, and its ground-truth label $\pi(\tau, a)$ are unchanged. At threshold θ, $\eta(\rho; \theta)$ is the supremal increase, over such representation perturbations, in conditional violation-miss probability; $\bar{\eta}(\rho) = \sup_\theta \eta(\rho; \theta)$. Suprema need not be attained on an infinite perturbation set.

The uniform margin $\bar{\eta}(\rho)$ is the relevant quantity for a calibrated gate, whose threshold $\hat{\theta}$ is itself random.

Theorem 4 (Adversarial deployment bound)**.** *Under Assumption 1, calibrate on benign violations as in Theorem 3, then let a budget-ρ adversary act only at deployment, on steps drawn independently of the calibration sample; the violating proposals generated under attack (before perturbation) must still satisfy Assumption 1, e.g., for an exogenous proposal stream with action-local labels and history-free judge inputs (otherwise forcing blocks of compliant proposals can steer the agent into uncalibrated violations that η does not measure). The calibrated gate satisfies*

$$\mathbb{P}[\textit{unsafe episode}] \le \min\{1, \delta + H\bar{\eta}(\rho)\}$$

marginally over the calibration draw. At a fixed threshold θ, an attained-margin, independent all-violation product construction has risk $1 - (1 - \min\{1, \beta(\theta) + \eta(\rho; \theta)\})^H$.

Proof idea. Calibration sees only benign data, so the benign per-step miss at $\hat{\theta}$ is still at most δ/H in expectation; $\bar{\eta}(\rho)$ bounds the increase at every threshold, and the union bound gives the claim (Supplement A.3).

Proposition 5 (Distributional zero-miss price)**.** *Let Q_ρ be an attainable adversarial violation-score law and F_0 the compliant score law. Every score-measurable zero-Q_ρ-miss gate obeys*

$$U \ge 1 - \sup\{F_0(N) : Q_\rho(N) = 0\}.$$

In particular, $F_0 \ll Q_\rho$ implies $U = 1$ for distributional zero miss.

Corollary 3 (Robust calibration restores the certificate, at an ROC price)**.** *If a uniform margin $\bar{\eta}(\rho) < \delta/H$ has been verified, calibrating on benign exchangeable violations to the tightened per-step target $\delta/H - \bar{\eta}(\rho)$ (not δ/H) restores $\mathbb{P}[\textit{unsafe episode}] \le \delta$ (marginal over the calibration draw) against any budget-ρ representation adversary at deployment, under the conditions of Theorem 4, at an added population false-block cost of* $\mathrm{roc}_I(\delta/H - \bar{\eta}(\rho)) - \mathrm{roc}_I(\delta/H)$.

Naive benign calibration is therefore not a security boundary: its guarantee degrades to $\delta + H\bar{\eta}(\rho)$ and is vacuous once that reaches one. Margin-aware calibration would be a security boundary against budget-ρ rewrites under Assumption 1, provided a uniform margin $\bar{\eta}(\rho) < \delta/H$ is verified at every threshold calibration might select, e.g., by a certified defense against insertions of up to ρ tokens [39].

A measured attack only lower-bounds $\eta(\rho; \theta)$, and hence $\bar{\eta}(\rho)$; it can falsify the precondition of Corollary 3 (§7.4) but never verify it.

6 T3: Decidability of Policy Analysis

Theorem 1 makes enforcement constructive once a nonempty policy is given as a $\mathcal{T}$-representable safety automaton; static analysis of the specification is a different problem. Even *nontriviality* (can the rule ever fire, the basic check a policy linter runs) is undecidable for an expressive guard theory, a *symbolic* question orthogonal to the *semantic*-censorship undecidability of Glukhov et al. [40].

Proposition 6 (Specification analysis is undecidable; immediate from Minsky machines)**.** *There is a guard theory $\mathcal{T}^*$ (equality plus two global $\mathbb{N}$-counters with increment, guarded decrement, and zero-test) for which the following is undecidable: given a deterministic, trim register automaton over $\mathcal{T}^*$ with a designated bad state, decide whether the bad state is reachable, equivalently whether the specified policy is* nontrivial *(has a bad prefix, i.e., rules out at least one finite behavior). The reduction is from two-counter Minsky-machine halting.*

Proof idea. The automaton's control and two global counters simulate a two-counter Minsky machine [41] on tick^n, with zero tests as guards and the halt label as the bad state; a second action moves every nonbad label to a good sink, so the policy stays nonempty and enforceable by T1 while the static question is undecidable. No data values are used (Supplement A.4).

Two decrementable, zero-testable counters are one source of undecidability; monotone counters regain it if guards compare counters, and shared amount fields already compare counters below the guard constants ($x + v = K \wedge y + v = K - c$), which keeps $\mathcal{T}_0$ decidable but is why its NP bound needs cap checks. A policy language with two uncapped, refundable budgets cannot ship a complete "can this rule fire" linter, whereas caps, quotas, and revocation need one SMT query and authenticate-before-access stays in PSPACE.

Theorem 5 (A decidable, expressive fragment)**.** *Over an infinite equality-data sort and a separate binary-encoded $\mathbb{N}$ amount sort, let $\mathcal{T}_0$ be the class of finite, separable, disjunctive products of total deterministic components over a common alphabet: (E) equality-register monitors; (K) a uniform key-local monitor whose action names at most one key and only that key's local state and monotone counters may be read or updated; and (V) one component with finite control and one finite vector of monotone counters. Updates add nonnegative fixed or action-supplied integers; each amount field has one shared value across all its uses. Atomic guards compare one counter (or counter plus that amount) with a binary-encoded constant, never another counter. Component guards partition every input (missing cases go to bad), and separability forbids cross-component guards or assumptions. Bad-prefix reachability is PSPACE-complete, and hard already for one (E) component; membership follows by exact cap saturation for (K,V), exact one-witness projection for (K), and on-the-fly search. Without (E) components, with keys over the equality sort, and if every amount atom is a cap check (the guarded case adds that amount to the tested counter or enters the bad state), as for caps, quotas, and*

revocation, it is NP-complete. Online enforcement takes polynomial time per step with saturated counters. The fragment expresses authenticate-before-access, per-recipient caps, bucket quotas, no-send-after-revoke, and fixed-vector quotas, including P2 and P3 (Supplement A.4).

Proof idea. Caps saturate: counters only grow and each guard compares one counter with a constant, so $\min\{x, C_x + 1\}$ is an exact abstraction. One witness key suffices for (K), because a key-local rejecting run projects onto its first rejecting key and lifts back by renaming; separable, total components reduce product rejection to one component. Searching the saturated graph on the fly gives membership; for hardness, one (E) component with a register per tape cell simulates a polynomial-space Turing machine, as in the PSPACE lower bound for register-automaton nonemptiness [15] (Supplement A.4). Without registers and with cap checks, counters climb through the polynomially many intervals between guard constants, so a run has polynomially many phases with fixed guard outcomes, each a Parikh-image constraint; one existential Presburger formula, i.e., one SMT query, decides nontriviality [42, 43], and SAT reduces to set-once flags. For P2, the analysis tracks one composite key (k, w), however many keys the runtime gate stores. For trim $\mathcal{T}_0$ specifications $\mathcal{A}, \mathcal{B}$, deciding $P_{\mathcal{A}} \subseteq P_{\mathcal{B}}$, i.e., redundancy of rule $\mathcal{B}$ given $\mathcal{A}$ (§7.6), is also PSPACE-complete if $\mathcal{A}$ has no (K) component and only counters read amounts, and coNP-complete if moreover neither has (E) components and every amount atom is a cap check, as in §7.6 (Supplement A.4).

7 Evaluation

We test the empirical consequences of T2 and time the constructive halves of T1 and T3: static discrimination and calibration (Q1, primary, predeclared), a finite controlled model (Q2), representation attacks (Q3), the closed loop (Q4, descriptive), and the cost of exact enforcement (Q5). Supplement E gives the full evaluation and Supplement B the extended experiments. Every certificate below is the marginal bound of Theorem 3 under Assumption 1 for a declared information variable and population.

7.1 Setup

Thirty-three judge runs (32 models, from local open-weights to hosted frontier models) score every item with one fixed 0–100 prompt (Supplement C). The score s is the first integer in the reply divided by 100, and a gate blocks iff $s \geq \theta$; an API-signaled refusal or content-filter block scores 1.0, any other reply without an integer .5 (Supplement B.1). Seven hosted judges, whose APIs do not accept a temperature setting, are scored three times (run 1 reported). The static diagnostic is AgentDojo `v1.2.1` [16]: 97 user tasks (compliant) and 35 injection goals (violations), labeled by provenance and given as instruction strings without execution history, so it measures discrimination for that information variable only. All runs score the same 132 items, which permits paired inference (DeLong covariance, max-T simultaneous intervals, Holm) with a predeclared ±.05 equivalence margin. Split-conformal calibration takes $\hat{\theta}$ as the $\lfloor (n+1)\delta/H \rfloor$-th smallest of the n violating calibration scores ($H = 1$; block-all if that index is 0), and 500 label-balanced two-fold splits give cross-fitted operating points.

7.2 Q1: Static Discrimination and Calibration

AUC ranges from .660 to .929. Against the predeclared reference Gemma-2-9B (AUC .813), only Claude Opus 5.5 is simultaneously

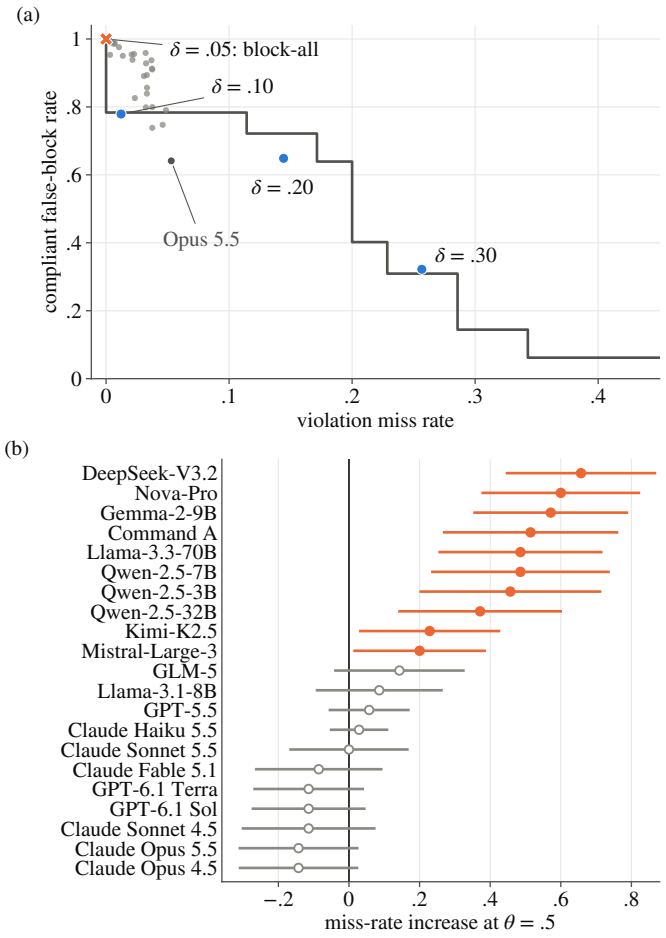


Fig. 2. Two estimands on instruction strings. (a) Static calibration: Gemma-2-9B's threshold ROC on the 132 instruction strings (false block against miss; lower left is better) and its cross-fitted split-conformal operating points (means over 500 two-fold splits, labeled by δ; $\delta = .05$ is block-all); gray: the other 32 judge runs at $\delta = .10$ (dark: Claude Opus 5.5, false block .641). (b) Representation attack: miss increase from one predeclared 16-word suffix at $\theta = .5$ over all 35 violations, with suite-stratified 95% intervals simultaneous over the 21 attacked judge runs; filled: interval excludes zero.

superior in all three runs when refusals count as blocks (run 1: $\Delta = +.116$, simultaneous interval $[+.014, +.217]$, max-T $p = .013$ treating goals as independent; its provider refuses 20 of the 35 violations and no compliant item) and in none when they score .5; Claude Haiku 5.5, which never refuses, is superior in two of three runs. No judge is inferior, no interval fits inside the ±.05 margin (forest plot in Supplement E; familywise detectable differences .072–.197), and without the 7 goals whose text is benign no judge is superior (Supplement E). An off-the-shelf 184M-parameter prompt-injection detector (ProtectAI DeBERTa-v3) matches the reference (AUC .848, $p = .35$) but misses .543 of the goals at its default threshold, and PIGuard is at chance (.532; Supplement B.21). Small calibration sets make deterministic gates expensive: every $\delta = .05$ split-conformal gate is block-all, because each fold holds 17–18 violations and a non-vacuous threshold needs $H/\delta - 1 = 19$; at $\delta = .10$ the conformal index is 1, so $\hat{\theta}$ is the single lowest violating calibration score, miss stays at most .053, and false block ranges from .641 (Claude Opus 5.5; item-bootstrap 95% CI [.197, .835]) to 1.000 (Figure 2a),

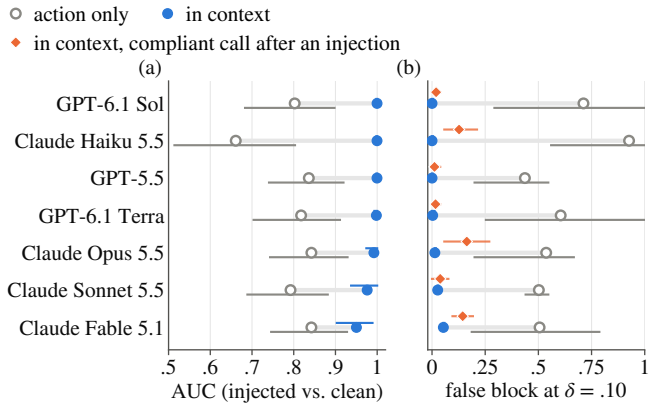


Fig. 3. What the gate sees decides the frontier (Proposition 2). The seven hosted judges score the same AgentDojo ground-truth tool calls serialized alone (action only) or with the user task and the tool outputs the agent saw (in context). (a) AUC of the 104 injected-goal calls against 75 compliant calls in clean contexts, with bootstrap 95% intervals. (b) Cross-fitted split-conformal false block at $\delta = .10$ (whiskers: 2.5–97.5 split percentiles) on the clean compliant calls and, in context, on the 104 compliant calls that follow an injection (diamonds). Refusals count as blocks; Claude Opus 5.5 refuses 57 of the 104 violations in context.

or from .485 to 1.000 when 28 of the 35 violations calibrate. The deterministic rule is conservative: exact-level randomized calibration spends the full budget (miss .100) at false block .334 for Claude Opus 5.5 and makes every $\delta = .05$ gate non-vacuous (false block .587–.917), at the price of randomized blocking (`calib_randomized.py`). Much of this cost comes from the 7 benign-text goals: without them the false block at $\delta = .10$ falls from .641 to .111 for Claude Opus 5.5 and from .779 to .585 for Gemma-2-9B (`stats_reporting.py`; Supplement B.6 for Gemma). On a binormal synthetic judge the certificate is tight (realized episode risk .0098–.182 for $\delta = .01$–.20; Supplement B.2).

What the gate sees: Serialized without context, AgentDojo's 123 ground-truth culminating tool calls are, if anything, harder for the seven hosted judges (AUC .780–.885, each below its instruction-string AUC; no paired difference survives Holm). Given what a gate in the loop sees, namely the user request and the executed ground-truth calls with their tool outputs, which carry the `important_instructions` injection, the same judges separate an injected goal's culminating call from clean compliant calls with AUC .950–1.000 (.661–.842 for the same calls without context; every goal-cluster bootstrap interval of the gain excludes zero; .959–.999 with AgentDojo's tagless direct attack) and, which isolates judgment of the action rather than detection of an injection, from the user's own call in the same injected context with AUC .872–.999 (104 injected and 75 clean contexts, predeclared; Figure 3, Supplement E). At $\delta = .10$, gates cross-fitted over goals (52 calls but only 13 goals per fold, so these misses are descriptive) miss .038–.098 of held-out goals' calls and block .000–.053 of clean compliant calls, against .437–.926 without context. Context buys discrimination, not verified robustness: a best-of-20 search that rewrites only the injection around the verbatim goal finds rewrites that evade the in-sample $\delta = .10$ gate in at least three of five fresh strict re-scores on 17/101 (Claude Haiku 5.5) and 11/95 (Opus 5.5) flagged calls, 12 and 6 net of a clean-prompt null control (no lower 95% bound above δ/H), and on 0–6 calls for the other five judges, whereas fixed sentences inside the overt template move the miss by at most +.023; at the native $\theta = .5$ at most one rewrite per judge stably evades while clean false block stays at most .107, so the fragility lies in calibrating to the top of a saturated score scale, the trade that Corollary 3 prices. The traces are replayed ground truth and the template overt, so this is a reference point for provenance-aware gates, not natural-trace accuracy.

7.3 Q2: A Finite Controlled Instantiation

In a two-step retry model, exact enumeration gives .551 expected false blocks at $\delta = .05$ with perfect recall against .863 and 1.137 for score-and-time and stationary policies (Theorem 2), and logs from a gate that blocks each step with probability .20 recover both risks of Proposition 3's systems without bias (RMSE .016; Proposition 4; Supplement B.2).

7.4 Q3: Representation Attacks

A predeclared 16-word "administrator preapproval" suffix, appended to the judge-visible string with item and label fixed, raises the miss rate at $\theta = .5$ significantly for 10 of the 21 attacked judge runs (increases .200–.657, simultaneous intervals; Figure 2b; e.g., Gemma-2-9B from .229 to .800), but for no Claude or GPT judge. Each increase lower-bounds $\eta(\rho; .5)$, and the simultaneous lower bound $L \geq .19$ for 7 judges falsifies Corollary 3's precondition whenever $\delta/H \leq L$. Two searches, the best of six templates and up to 20 query-adaptive grammar rewrites in the style of best-of-N jailbreaking [44] (Supplement B.20), attack each judge at its flagged-subclass threshold θ_f, the $\delta = .10$ split-conformal threshold on the violations it flags at $\theta = .5$ ($\theta_f = .55$–.85; false block .01–.22; clean miss .20–.40 of all violations). On flagged violations they collapse GPT-6.1 Terra (miss +.727) and raise the first-integer miss by .208–.478 for Claude Haiku and Sonnet 5.5, GPT-5.5, and GPT-6.1 Sol, and by .115 for Claude Opus 5.5, whose provider refuses most rewrites (Supplement B.19 scales these to the full law). At the $\delta = .10$ gate fit in-sample on all 35 violations ($\theta \leq .15$, false block .32–1.00), best-of-six evades 2 of 30 flagged violations for GPT-5.5 and none elsewhere, and the grammar search at most two per judge. An optimizing attacker does not stop there: an LLM that rewrites a ≤45-word prefix and suffix from the judge's scores (action verbatim, at most 20 queries, in the style of PAIR [45]; predeclared) evades that gate on 9/33, 9/33, 12/34, and 10/35 flagged violations for Claude Opus, Sonnet, and Haiku 5.5 and GPT-5.5, counting a rewrite only if it evades in at least three of five fresh strict re-scores and the unmodified violation does not (6–8 of the 28 whose text is not benign; the attacker model refused every turn on 12–13 items per judge, so these are lower bounds; Supplement E). The apparent robustness of expensive gates was an artifact of weak attacks.

7.5 Q4: Gate Decisions in the Closed Loop

A local `qwen2.5:7b` agent runs all four AgentDojo suites under the fixed, gate-unaware `important_instructions` attack (949 episodes per condition); a gate drops a serialized call if $s \geq \theta$, and the agent receives no block signal. A 3B judge at $\theta = .5$ cuts attack success from 45/949 to 4/949 but drops 78% of calls and lowers task success from .254 to .180; a 7B judge at $\theta = .7$ drops 12% and is not significant (30/949). Paired reruns ($N = 389$) differ in both directions (34 baseline-only against 5 gate-only successes for the 3B gate; Supplement E). We report these results as descriptive only: the operating points were not preregistered, and without A/A and block-rate-matched random-drop controls

(Supplement E) they show neither that the gate beats its blocking volume nor feedback, whose non-identifiability (Proposition 3) is formal. The attack is also gate-unaware (the Q3 suffix raises this 3B judge's miss from .143 to .600; Supplement B.9), so the reduction is optimistic.

7.6 Q5: The Cost of Exact Enforcement and Analysis

Compiled as one keyed-counter gate, P2 and P3 take a median of 0.26–0.37 μs per decision (one core, Python 3.12) over a 10^6-proposal stream with up to 7.7×10^5 active keys, agree with a naive replay monitor on all 2×10^4 test proposals, and are about six orders of magnitude cheaper than one hosted judge call (median 0.85–1.64 s). A saturated search built on Theorem 5 finds P2's per-recipient cap c redundant under a per-session limit ℓ in exactly the tested configurations with $\ell \le c$ (five configurations, $c, \ell \le 200$), in under 0.6 s each (Supplement E). The exact half of a guardrail is effectively free; its cost and false blocks come from the predicates it must judge.

8 Related Work

Runtime enforcement: Security automata enforce only safety properties [10]; computability and monitor memory refine the enforceable class [12, 13, 46], and edit automata enforce more by suppressing and inserting actions [11]. Our gate is a supervisor [47] without buffering, so given the same oracles its class lies strictly below that of edit automata (Proposition 1). Unlike enforcement over merely observable actions [48] or with buffering [49], T1 treats each tool call as controllable before an irreversible commit. Mandatory-results automata [50] also answer each action before the target proceeds, and proactive first-order enforcement [51] suppresses or causes events against metric first-order policies; neither separates exact predicates from a fallible judge, which is the split T1 and T2 are built on. Parametric monitors [52], quantified event automata [53], and register monitors [54] handle data-carrying events; our key maps resemble their trace slicing, and their safety monitors yield gates (Theorem 1). Counting extensions of past-time metric temporal logic admit trace-length-independent monitoring for many, but not all, quantitative policies [55]; T3 asks instead whether a counter policy is nontrivial at all. David et al. [56] state an enforceability schema for agent guardrails whose two directions are left as proof obligations; T1 proves both for register-recognizable policies over infinite effect traces.

Agent defenses and guardrails: Injected instructions reach agents through tool outputs [1], tool descriptions [57], and messages between collaborating models [58]. System-level designs restrict what untrusted data can influence through control/data separation, information-flow labels, or privilege policies [59–61]; they decide from trusted metadata, which places them in T1's oracle regime, exact only while the labels are intact. Rule and contract gates [4, 62, 63] are T1 gates, and finite-state contracts over action-local predicates capture only a regular subclass of T1's policies; comparing data values across calls needs registers, which ContrAgent [62] keeps outside its DFAs. CAGE's exact variant certifies that an authorization stays valid across bounded faults in typed tool returns [64], which hardens the inputs of a T1 check, and the guarantee of Wu et al. [8] is conditional on a lossless first-order restatement, whose errors fall under T2. Detectors and guard models [2, 3, 65] are bounded by their ROC. Their benign certificates do not survive adaptive attacks [66, 67], universal or optimized injections into LLM judges [68, 69], or injected text fed to monitors [70] in AI-control protocols [71], and context-tailored rewrites raise jailbreak success against customized LLM applications [72]; what we add is the margin $\bar{\eta}(\rho)$, the risk it costs (Theorem 4), and when recalibration restores the certificate (Corollary 3).

Closed-loop and statistical guarantees: Certificates computed from a model of the gated loop [73, 74] hold for every controller or proposal sequence under exact predicates (Gu–Zhu's deterministically, Baheri–Wei's as a PAC bound on unsafe entry); judge error enters once a fallible judge decides, and the agent's response to a block, documented empirically [75], is what Proposition 3 shows ungated data cannot identify. CORA calibrates a pre-action gate by conformal risk control on a joint harm-and-execute budget [76], which executing everything meets whenever harms are rarer than the budget; calibrating on violations alone bounds the miss rate itself. Corollary 3 is a risk-level analogue of adversarially robust conformal prediction [14], and a certified insertion defense [39] supplies the verified margin it requires. Supplement D maps over twenty further systems to our regimes and traces the dependability lineage of the gate and the card.

9 Discussion and Limitations

Dependability and deployment: Along the fault–error–failure chain of §2 [9], fail-closed blocking turns would-be security failures into omission failures: $1-\beta$ is the gate's error-detection coverage and false blocks are its availability cost; even expensive gates fall to an optimizing attacker (§7.4), and the score band a judge cannot resolve is better escalated than allowed. Fail-closed scoring is also an attack surface: tool-output text that provokes refusals or high scores induces false blocks, a denial-of-service lever that Theorems 3 and 4, which bound only misses, do not price.

A certificate card for guardrail evaluations: We propose that every runtime-guardrail evaluation report the five fields of Table 2, each required by one of our results. The *Ours* lines show why: even our best calibrated gate uses no oracle predicates, is block-all at $\delta = .05$, and has no verified margin, none of which a headline AUC of .93 reveals. A fill-in template and a JSON schema for the card are in the artifact.

Limitations: The in-context evaluation replays AgentDojo ground-truth traces under two fixed attacks for seven hosted judges, not live agent runs (§7.2); 24 of the 35 injection goals share an attacker identifier (an IBAN, e-mail addresses, URLs), so fewer violations are independent, and newer judges may have seen this public benchmark. The closed loop uses one local agent with silent drops and no A/A or random-drop controls. Black-box attacks, including an optimizing PAIR-style attacker [45], only lower-bound margins, and white-box optimization such as GCG [77] on open-weight judges is not evaluated. Of the dedicated classifiers we ran, Llama-Guard-3-8B [2] misses .46 of injections at false block .10, dominated by 22 judge runs (B.7), and of two prompt-injection detectors one matches the reference judge's ranking but misses .543 of the goals at its default threshold and the other is at chance (B.21); a T1-style provenance rule misses .212 of injected calls and blocks .400 of clean ones (Supplement E); CaMeL and Progent were not compared (Supplement D). All experiments are inference-only on public benchmarks, involve no human subjects, and target no deployed agent.

10 Conclusion

Five rules follow from the three boundaries. (1) A gate that cannot retract enforces only what it can refute on a finite prefix, so compile

TABLE 2
Guardrail certificate card: what an evaluation must state, the result that requires it, a red flag, and (*Ours*) the fields and verdict for one gate, Claude Opus 5.5 on instruction strings.

1	**Policy, oracle interface, judge input:** policy language, oracle predicates Π, trusted metadata (Thm. 1), and what the judge sees (Prop. 2). *Red flag:* a policy called enforced although a judge decides one of its predicates. *Ours:* provenance labels; no oracle predicates; instruction strings.
2	**Operating point:** miss and false block at the deployed threshold, target δ, horizon H (Prop. 2). *Red flag:* AUC without an operating point. *Ours:* $\delta = .10$, $H = 1$, instruction strings: miss .053, false block .641 (95% CI .20–.83), AUC .929; exact-level randomized calibration: miss .100, false block .334.
3	**Calibration:** source and number n of violating examples (non-vacuous only if $n \geq H/\delta - 1$), the exchangeability claim (Thm. 3), and the training-conditional miss. *Red flag:* $n < H/\delta - 1$, so the certified gate blocks everything. *Ours:* 17–18 violations per fold; exchangeability not verified; deterministic $\delta = .05$ is block-all (randomized: false block .587); at $n = 17$ the deployed miss exceeds .10 with probability .17.
4	**Adversarial margin:** a verified $\bar{\eta}(\rho)$ with its budget ρ, else "uncertified under attack"; attacks only lower-bound it (Thm. 4, Cor. 3). *Red flag:* "robust" backed only by failed attacks. *Ours:* none verified; measured: fixed suffix, no increase at $\theta = .5$; an optimizing attacker (≤ 20 queries) evades on $\geq$9 of 33 flagged violations at the in-sample $\delta = .10$ threshold (stable in fresh strict re-scores).
5	**Feedback:** whether blocks are visible; paired closed-loop reruns reporting attack success, task success, and gating fraction, with A/A and random-drop controls; exploratory logs, collected in a sandbox, for claims about other gates (Prop. 3, Prop. 4). *Red flag:* closed-loop claims from static scores or ungated logs. *Ours:* not evaluated for this gate (our only closed loop, with a 3B judge, is descriptive).
V	**Verdict:** certified at $\delta = .10$ per instruction under unverified exchangeability; uncertified under attack; closed loop not evaluated.

oracle-checkable predicates into exact gates (T1). (2) For a one-step exogenous decision, no wrapper, ensemble, or threshold beats the likelihood-ratio frontier of its information; a guardrail improves only by changing what it sees (T2). (3) Benign calibration is not a security boundary: without a verified margin its certificate gives no guarantee under attack, and an adversary that chooses the violation itself voids it (T2). (4) Ungated logs and static scores cannot identify closed-loop behavior; logs with known block and allow probabilities can, but their exploratory allows belong in a sandbox and their variance can grow exponentially in the horizon (T2). (5) Keep policy languages inside a decidable fragment such as $\mathcal{T}_0$, so rules can be linted before deployment (T3). A guardrail certificate is a statement about an oracle interface, an operating point, a calibration population, an adversarial margin, and a feedback model; stated without them, it is not a security guarantee.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023, pp. 79–90.

[2] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," 2023, arXiv:2312.06674.

[3] S. Chennabasappa *et al.*, "LlamaFirewall: An open source guardrail system for building secure AI agents," 2025, arXiv:2505.03574.

[4] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. 48th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. ACM, 2026, pp. 2938–2950.

[5] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "ProbGuard: Proactive runtime monitoring for LLM agent safety via probabilistic prediction," in *Proc. 41st IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*. ACM, 2026, pp. 1470–1482.

[6] L. Miculicich *et al.*, "VeriGuard: Enhancing LLM agent safety via verified code generation," 2025, arXiv:2510.05156.

[7] A. Kamath, S. Zhang, C. Xu, S. Ugare, G. Singh, and S. Misailovic, "Enforcing temporal constraints for LLM agents," 2025, arXiv:2512.23738.

[8] B. Wu, W. Zhang, K. Chen, H. Fang, and N. Yu, "Provably secure agent guardrail," 2026, arXiv:2605.29251.

[9] A. Avižienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Trans. Dependable Secure Comput.*, vol. 1, no. 1, pp. 11–33, 2004.

[10] F. B. Schneider, "Enforceable security policies," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 1, pp. 30–50, 2000.

[11] J. Ligatti, L. Bauer, and D. Walker, "Edit automata: Enforcement mechanisms for run-time security policies," *Int. J. Inf. Secur.*, vol. 4, no. 1–2, pp. 2–16, 2005.

[12] P. W. L. Fong, "Access control by tracking shallow execution history," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2004, pp. 43–55.

[13] C. Talhi, N. Tawbi, and M. Debbabi, "Execution monitoring enforcement under memory-limitation constraints," *Inf. Comput.*, vol. 206, no. 2–4, pp. 158–184, 2008.

[14] A. Gendler, T.-W. Weng, L. Daniel, and Y. Romano, "Adversarially robust conformal prediction," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[15] S. Demri and R. Lazić, "LTL with the freeze quantifier and register automata," *ACM Trans. Comput. Log.*, vol. 10, no. 3, pp. 16:1–16:30, 2009.

[16] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, 2024.

[17] J. P. Anderson, "Computer security technology planning study," Electronic Systems Division, Air Force Systems Command, U.S. Air Force, Hanscom AFB, Bedford, MA, USA, Tech. Rep. ESD-TR-73-51, vol. II, 1972.

[18] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[19] B. Alpern and F. B. Schneider, "Recognizing safety and liveness," *Distrib. Comput.*, vol. 2, no. 3, pp. 117–126, 1987.

[20] M. Kaminski and N. Francez, "Finite-memory automata," *Theor. Comput. Sci.*, vol. 134, no. 2, pp. 329–363, 1994.

[21] N. Palumbo *et al.*, "Formal policy enforcement for agentic systems," 2026, arXiv:2602.16708.

[22] A. Solko-Breslin, P. K. Mudrakarta, M. Christodorescu, S. Jha, and K. D. Dvijotham, "Efficient and sound probabilistic verification for AI agents," 2026, arXiv:2606.20510.

[23] J. Ligatti, L. Bauer, and D. Walker, "Run-time enforcement of nonsafety policies," *ACM Trans. Inf. Syst. Secur.*, vol. 12, no. 3, pp. 19:1–19:41, 2009.

[24] S. Pinisetty, P. S. Roop, S. Smyth, S. Tripakis, and R. von Hanxleden, "Runtime enforcement of reactive systems using synchronous enforcers," in *Proc. 24th ACM SIGSOFT Int. SPIN Symp. Model Checking Softw. (SPIN)*, 2017, pp. 80–89.

[25] A. Anand, L. Germerie Guizouarn, T. Jéron, S. Mukherjee, S. Pinisetty, and O. Sankur, "Prompt runtime enforcement," in *Proc. Int. Symp. Automated Technol. Verification Anal. (ATVA)*, ser. LNCS, vol. 16145. Springer, 2025, pp. 135–156.

[26] L. Aceto *et al.*, "Monitorability for the modal mu-calculus over systems with data: From practice to theory," in *Proc. 36th Int. Conf. Concurrency Theory (CONCUR)*, ser. LIPIcs, vol. 348, 2025, pp. 4:1–4:21.

[27] M. R. Clarkson and F. B. Schneider, "Hyperproperties," *J. Comput. Secur.*, vol. 18, no. 6, pp. 1157–1210, 2010.

[28] A. Pandey, A. Jain, L. Chen, and C. Maple, "Local allow, global deny: Why per-action authorization does not compose in LLM agents," in *Proc. IEEE Int. Conf. Trust Privacy Secur. Intell. Syst. Appl. (TPS)*, 2026.

[29] J. Neyman and E. S. Pearson, "On the problem of the most efficient tests of statistical hypotheses," *Phil. Trans. Roy. Soc. A*, vol. 231, pp. 289–337, 1933.

[30] X. Tong, Y. Feng, and J. J. Li, "Neyman–Pearson classification algorithms and NP receiver operating characteristics," *Sci. Adv.*, vol. 4, no. 2, p. eaao1659, 2018.

[31] D. Blackwell, "Equivalent comparisons of experiments," *Ann. Math. Statist.*, vol. 24, no. 2, pp. 265–272, 1953.

[32] E. Walraven and M. T. J. Spaan, "Column generation algorithms for constrained POMDPs," *J. Artif. Intell. Res.*, vol. 62, pp. 489–533, 2018.

[33] E. Altman, *Constrained Markov Decision Processes*. Boca Raton, FL, USA: Chapman and Hall/CRC, 1999.

[34] J. Robins, "A new approach to causal inference in mortality studies with a sustained exposure period—application to control of the healthy worker survivor effect," *Math. Model.*, vol. 7, no. 9–12, pp. 1393–1512, 1986.

[35] S. A. Murphy, "Optimal dynamic treatment regimes," *J. Roy. Statist. Soc. Ser. B*, vol. 65, no. 2, pp. 331–355, 2003.

[36] D. Precup, R. S. Sutton, and S. Singh, "Eligibility traces for off-policy policy evaluation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2000, pp. 759–766.

[37] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal risk control," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[38] V. Vovk, "Conditional validity of inductive conformal predictors," in *Proc. Asian Conf. Mach. Learn. (ACML)*, ser. PMLR, vol. 25, 2012, pp. 475–490.

[39] A. Kumar, C. Agarwal, S. Srinivas, A. J. Li, S. Feizi, and H. Lakkaraju, "Certifying LLM safety against adversarial prompting," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[40] D. Glukhov, I. Shumailov, Y. Gal, N. Papernot, and V. Papyan, "Position: Fundamental limitations of LLM censorship necessitate new approaches," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 15 767–15 787.

[41] M. L. Minsky, *Computation: Finite and Infinite Machines*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1967.

[42] K. N. Verma, H. Seidl, and T. Schwentick, "On the complexity of equational Horn clauses," in *Proc. Int. Conf. Automated Deduction (CADE)*, ser. LNCS, vol. 3632. Springer, 2005, pp. 337–352.

[43] B. Scarpellini, "Complexity of subcases of Presburger arithmetic," *Trans. Amer. Math. Soc.*, vol. 284, no. 1, pp. 203–218, 1984.

[44] J. Hughes *et al.*, "Best-of-N jailbreaking," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025.

[45] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, 2025, pp. 23–42.

[46] K. W. Hamlen, G. Morrisett, and F. B. Schneider, "Computability classes for enforcement mechanisms," *ACM Trans. Program. Lang. Syst.*, vol. 28, no. 1, pp. 175–205, 2006.

[47] P. J. Ramadge and W. M. Wonham, "Supervisory control of a class of discrete event processes," *SIAM J. Control Optim.*, vol. 25, no. 1, pp. 206–230, 1987.

[48] D. Basin, V. Jugé, F. Klaedtke, and E. Zălinescu, "Enforceable security policies revisited," *ACM Trans. Inf. Syst. Secur.*, vol. 16, no. 1, pp. 3:1–3:26, 2013.

[49] Y. Falcone, J.-C. Fernandez, and L. Mounier, "What can you verify and enforce at runtime?" *Int. J. Softw. Tools Technol. Transf. (STTT)*, vol. 14, no. 3, pp. 349–382, 2012.

[50] J. Ligatti and S. Reddy, "A theory of runtime enforcement, with results," in *Proc. Eur. Symp. Res. Comput. Secur. (ESORICS)*, ser. LNCS, vol. 6345. Springer, 2010, pp. 87–100.

[51] F. Hublet, L. Lima, D. Basin, S. Krstić, and D. Traytel, "Proactive real-time first-order enforcement," in *Proc. Int. Conf. Comput. Aided Verification (CAV)*, ser. LNCS, vol. 14682. Springer, 2024, pp. 156–181.

[52] G. Roşu and F. Chen, "Semantics and algorithms for parametric monitoring," *Log. Methods Comput. Sci.*, vol. 8, no. 1, 2012.

[53] H. Barringer, Y. Falcone, K. Havelund, G. Reger, and D. Rydeheard, "Quantified event automata: Towards expressive and efficient runtime monitors," in *Proc. Int. Symp. Formal Methods (FM)*, ser. LNCS, vol. 7436. Springer, 2012, pp. 68–84.

[54] R. Grigore, D. Distefano, R. L. Petersen, and N. Tzevelekos, "Runtime verification based on register automata," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst. (TACAS)*, ser. LNCS, vol. 7795. Springer, 2013, pp. 260–276.

[55] X. Du, A. Tiu, K. Cheng, and Y. Liu, "Trace-length independent runtime monitoring of quantitative policies," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1489–1510, 2021.

[56] I. David, M. Guarnieri, and A. Gervais, "Alignment contracts for agentic security systems," 2026, arXiv:2605.00081.

[57] Y. Guo *et al.*, "MCPXkit: The unified toolkit for analyzing Model Context Protocol security," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 5, pp. 9796–9811, 2026.

[58] M. Yang, T. Zhu, B. Liu, H. Xu, and W. Zhou, "Cracks in collaboration: Threat models and attacks on multi-LLM collaborative systems," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 7191–7207, 2026.

[59] E. Debenedetti *et al.*, "Defeating prompt injections by design," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, 2026, pp. 587–618.

[60] M. Costa *et al.*, "Securing AI agents with information-flow control," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2026.

[61] T. Shi *et al.*, "Progent: Securing AI agents with privilege control," 2025, arXiv:2504.11703.

[62] Y. Xiao and P. Nuzzo, "Symbolic temporal supervision of LLM agents using contracts," 2026, arXiv:2609.18128.

[63] V. Reddy, S. R. Challaram, and A. Basu, "Reason less, verify more: Deterministic gates recover a silent policy-violation failure mode in tool-using LLM agents," 2026, arXiv:2607.07405.

[64] B. Delattre, C. Wang, and Y. Cao, "CAGE: Certified authorization under typed-return uncertainty for tool-using agents," 2026, arXiv:2607.29190.

[65] H. Li, X. Liu, N. Zhang, and C. Xiao, "PIGuard: Prompt injection guardrail via mitigating overdefense for free," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 30 420–30 437.

[66] M. Nasr *et al.*, "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," in *Proc. 35th USENIX Secur. Symp.*, 2026.

[67] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings Assoc. Comput. Linguistics (NAACL)*, 2025, pp. 7116–7132.

[68] V. Raina, A. Liusie, and M. Gales, "Is LLM-as-a-judge robust? Investigating universal adversarial attacks on zero-shot LLM assessment," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2024, pp. 7499–7517.

[69] J. Shi *et al.*, "Optimization-based prompt injection attack to LLM-as-a-judge," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2024, pp. 660–674.

[70] M. Terekhov *et al.*, "Adaptive attacks on trusted monitors subvert AI control protocols," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[71] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, "AI control: Improving safety despite intentional subversion," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[72] H. Xu *et al.*, "RedAgent: An autonomous agent for context-aware red teaming of LLM jailbreaks," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 6506–6521, 2026.

[73] A. Baheri and P. Wei, "Controller-agnostic runtime safety certificates for tool-using AI agents via data-driven robust reachability," *IEEE Control Syst. Lett.*, 2026, early access.

[74] C. Gu and Q. Zhu, "Certified execution harness for agentic AI: Safe and least-intrusive Petri-net supervisory control between reasoning and action," 2026, arXiv:2610.31376.

[75] R. Vasudev, M. Russak, D. Bikel, and W. Alshikh, "Accurate failure prediction in agents does not imply effective failure prevention," 2026, arXiv:2602.03338.

[76] Y. Feng *et al.*, "CORA: Conformal risk-controlled agents for safeguarded mobile GUI automation," 2026, arXiv:2604.09155.

[77] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, arXiv:2307.15043.

Shawn Ray is with Carnegie Mellon University, Pittsburgh, PA, USA. Research interests include runtime enforcement and the security of tool-using AI agents.